

MACHINE LEARNING

CẨM NANG ĐẦY ĐỦ



Thuật toán &
Khái niệm



Mô hình &
Huấn luyện



Supervised &
Unsupervised Learning



Đánh giá &
Metrics



Thực hành &
Dự án

TỪ CƠ BẢN ĐẾN NÂNG CAO

LƯỢT SANG TRANG SAU →

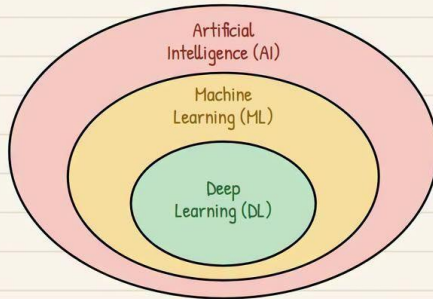
Kiến thức chất lượng, sự nghiệp tốt hơn!



Nền Tảng Machine Learning

1

① AI vs ML vs Deep Learning



Mối quan hệ phân cấp - DL là tập con của ML, ML là tập con của AI.

→ Giúp máy móc có thể suy nghĩ và hành động như con người.

→ Giúp máy móc học từ dữ liệu và tự cải thiện mà không cần lập trình tường minh.

→ Sử dụng neural network với nhiều lớp để học các pattern phức tạp từ lượng lớn dữ liệu.

② Machine Learning là gì?

ML là lĩnh vực nghiên cứu giúp máy tính có khả năng học từ dữ liệu và đưa ra dự đoán hoặc quyết định mà không cần được lập trình một cách tường minh.

★ Ý Tưởng Chính

Học các pattern từ dữ liệu → Tổng quát hoá → Đưa ra dự đoán trên dữ liệu mới, chưa từng thấy.

③ Lập Trình Truyền Thống vs Machine Learning

Lập Trình Truyền Thống



Rules do con người viết ra.

Hoạt động tốt khi ta đã biết rõ rules.

Machine Learning



Model tự học rules/pattern từ dữ liệu.

Hoạt động tốt khi rules chưa rõ hoặc quá phức tạp.

④ Máy Học Như Thế Nào?

① Thu Thập Dữ Liệu



Thu thập dữ liệu liên quan và chất lượng tốt.

② Chuẩn Bị Dữ Liệu



Làm sạch, xử lý missing values, encode, scale, feature engineering.

③ Huấn Luyện Model



Model học pattern từ training data.

④ Đánh Giá Model



Kiểm tra hiệu suất trên validation/test data.

⑤ Đưa Ra Dự Đoán



Dùng model để dự đoán dữ liệu mới, chưa từng thấy.

⑤ Ứng Dụng Thực Tế

Tài Chính

Phát hiện gian lận, chấm điểm tín dụng, giao dịch thuật toán.

Thương Mại Điện Tử

Hệ thống gợi ý, phân khúc khách hàng, dự báo nhu cầu.

Y Tế

Dự đoán bệnh, chẩn đoán hình ảnh y khoa, chấm điểm rủi ro bệnh nhân.

Ô Tô

Xe tự lái, bảo trì dự đoán, tối ưu tuyến đường.

Truyền Thông

Phát hiện spam, phân tích cảm xúc, chatbot.

Công Nghiệp

Kiểm tra chất lượng, phát hiện lỗi, tối ưu quy trình.

✓ Ghi Nhớ

- ML học pattern, không phải rules được lập trình cứng.
- Dữ liệu chất lượng > Model phức tạp.
- ML tốt = Dữ liệu tốt + Đặt vấn đề tốt.

💡 Mẹo Phòng Vấn

Luôn bắt đầu bằng việc hiểu rõ bài toán, dữ liệu và mục tiêu. Model chỉ là một phần của giải pháp!

Kiến thức chất lượng, sự nghiệp tốt hơn!



Các Loại Machine Learning

2

Các thuật toán Machine Learning (ML) học pattern từ dữ liệu.
Dựa trên loại dữ liệu & phản hồi, ML được chia thành 4 nhóm chính.

1	2	3	4
Supervised Learning	Unsupervised Learning	Semi-Supervised Learning	Reinforcement Learning
Model học từ dữ liệu đã gắn nhãn (Labeled data). Ta đã biết trước output đúng (target).	Model tìm pattern hoặc cấu trúc trong dữ liệu chưa gắn nhãn. Không có output đúng cho trước.	Kết hợp một lượng nhỏ dữ liệu đã gắn nhãn với lượng lớn dữ liệu chưa gắn nhãn.	Model (agent) học bằng cách tương tác với environment và nhận reward hoặc penalty.
Ví dụ - Phát hiện Email Spam - Dự đoán giá nhà - Chấm điểm rủi ro tín dụng - Phân loại ảnh Khi nào dùng? Dùng khi ta có dữ liệu lịch sử đã gắn nhãn và muốn dự đoán một output cụ thể.	Ví dụ - Phân khúc khách hàng - Market Basket Analysis - Topic Modeling - Phát hiện bất thường Khi nào dùng? Dùng khi dữ liệu chưa gắn nhãn và ta muốn khám phá cấu trúc hoặc nhóm dữ liệu.	Ví dụ - Phân loại ảnh - Nhận diện giọng nói - Chẩn đoán y khoa - Phân loại văn bản Khi nào dùng? Dùng khi dữ liệu gắn nhãn đắt đỏ hoặc khó thu thập, nhưng dữ liệu chưa gắn nhãn lại dồi dào.	Ví dụ - Chơi Game (Atari, Chess) - Điều hướng robot - Xe tự lái - Hệ thống gợi ý Khi nào dùng? Dùng khi có chuỗi quyết định tuần tự và phản hồi được đưa ra dưới dạng reward.

So Sánh Nhanh

Loại	Dữ Liệu	Nhãn	Mục Tiêu	Ví Dụ	Phù Hợp Cho
Supervised Learning	Labeled Data	✓	Dự đoán output	Spam Detection, House Price	Dự đoán & phân loại
Unsupervised Learning	Unlabeled Data	X	Tìm pattern/cấu trúc	Customer Segmentation	Phân tích khám phá
Semi-Supervised Learning	Labeled + Unlabeled	Một phần	Cải thiện học với ít nhãn	Image Classification	Khi nhãn bị giới hạn
Reinforcement Learning	Interaction Data	Không	Tối đa hoá reward tích lũy	Game Playing, Robotics	Bài toán quyết định tuần tự

✓ Ghi Nhớ

Chọn đúng loại dựa trên dữ liệu, bài toán và mục tiêu của bạn!

⚠ Lỗi Thường Gặp

Dùng thuật toán supervised trên dữ liệu chưa gắn nhãn.

💡 Mẹo Phòng Vấn

Nêu rõ loại ML, lý do bạn chọn nó và cách nó phù hợp với bài toán.

Điểm Chính Cần Nhớ

- ✓ Supervised → Dự đoán
- ✓ Unsupervised → Khám phá
- ✓ Semi-Supervised → Tận dụng nhãn giới hạn
- ✓ Reinforcement → Học qua thử-sai & reward

Kiến thức chất lượng, sự nghiệp tốt hơn!



Nền Tảng Dữ Liệu & ML

3

Dữ liệu chất lượng cao là nền tảng của mọi mô hình ML thành công.
Hãy cùng tìm hiểu những thành phần cốt lõi.

1 Thuật Ngữ Chính

- Dataset** : Tập hợp các bản ghi dữ liệu.
Data Point / Sample : Một bản ghi đơn lẻ trong dataset.
Feature : Biến đầu vào (X) dùng để mô tả pattern.
Label : Giá trị đầu ra (Y) gắn với một data point.
Target Variable : Label mà ta muốn model dự đoán.

Ví dụ Dataset (Customer Churn)

ID	Age	Tenure (months)	Monthly Charges (\$)	Contract Type	Churn (Target)
1	28	12	29.50	Month-to-month	1
2	45	24	56.70	One year	0
3	34	6	25.10	Month-to-month	1
...	...	...	...	...	...

-- Features (X) --

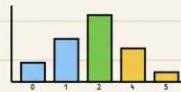
Label (Y)

2 Các Loại Dữ Liệu

Numerical Data

Giá trị là các con số.

- Discrete (đếm được)
- Continuous (đo lường được)



(ví dụ: Age, Income, Temperature)

Categorical Data

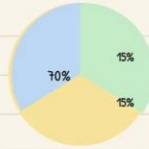
Giá trị là các category hoặc nhãn.

- Nominal (không có thứ tự)
- Ordinal (có thứ tự)

(ví dụ: City, Gender, Education Level)

Nominal: Red, Blue, Green
Ordinal: Low, Medium, High

3 Train / Validation / Test Split



Training Set (70%)

Dùng để train model. Model học pattern từ dữ liệu này.

Validation Set (15%)

Dùng để tune hyperparameter và chọn model tốt nhất.

Test Set (15%)

Chỉ dùng một lần để đánh giá hiệu suất model cuối cùng.

Vì Sao Phải Split?

- ✓ Giúp ngăn Overfitting
- ✓ Đảm bảo model tổng quát hoá tốt
- ✓ Đánh giá khách quan trên dữ liệu chưa từng thấy

4 Cấu Trúc Một Data Point

Feature 1	Feature 2	...	Feature n	Target (Label)
Age = 30	Income = 45K	...	Tenure = 12	Churn = 1

Input Features (X): giúp model hiểu pattern. | Target/Label (Y): điều model cần dự đoán.

Ví Dụ Thực Tế

Dự Đoán Customer Churn

Dự đoán liệu một khách hàng có rời bỏ dịch vụ hay không.

- Features: Age, Tenure, Usage, Monthly Charges, Contract Type, ...
- Target: Churn (0 = No, 1 = Yes)

⚠ Mẹo Phòng Vấn

Dữ liệu tốt >> Model tốt. Hầu hết các dự án ML dành 70-80% thời gian cho việc chuẩn bị dữ liệu.

✓ Best Practices

- Hiểu dữ liệu trước khi modeling.
- Xử lý missing values và outliers.
- Tránh Data Leakage bằng mọi giá.

⚠ Lỗi Thường Gặp

- Dùng test set trong lúc training.
- Bỏ qua missing values.
- Xây model trên dữ liệu chưa sạch.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Tiền Xử Lý Dữ Liệu

4

Tiền xử lý dữ liệu là quá trình làm sạch và chuẩn bị dữ liệu thô để có thể dùng cho việc huấn luyện các mô hình **Machine Learning** hiệu quả.

1 Các Bước Chính



Missing Values

Xử lý giá trị thiếu hoặc null bằng các chiến lược như mean, median, mode hoặc drop.



Duplicates

Loại bỏ các dòng trùng lặp để tránh kết quả bị lệch.



Outliers

Phát hiện và xử lý giá trị cực trị bằng IQR, Z-Score hoặc domain knowledge.



Categorical Encoding

Chuyển categorical data sang dạng số. (Label Encoding / One-Hot Encoding)



Feature Scaling

Scale các feature về cùng range để model hoạt động tốt hơn. (Normalization / Standardization)



Train-Test Split

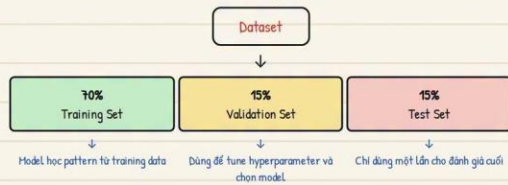
Chia dữ liệu thành training, validation và test set trước khi train model.



Data Leakage

Ngăn thông tin từ test data rò rỉ vào quá trình training.

2 Train-Validation-Test Split



Tip

Luôn split dữ liệu trước khi thực hiện bất kỳ bước tiền xử lý nào để tránh data leakage.

3 Kỹ Thuật Scaling

Normalization (Min-Max Scaling)

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

Scale giá trị về khoảng [0,1]. Hữu ích khi dữ liệu không phân phối chuẩn.

Standardization (Z-Score Scaling)

$$x' = (x - \mu) / \sigma$$

Biến đổi dữ liệu về mean=0, std=1. Hữu ích khi dữ liệu phân phối chuẩn.

4 Ví Dụ Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load data
df = pd.read_csv('data.csv')

# Handle missing values
df.fillna(df.mean(numeric_only=True), inplace=True)

# One-Hot Encoding
df = pd.get_dummies(df, drop_first=True)

# Features and Target
X = df.drop('target', axis=1)
y = df['target']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Feature Scaling (Standardization)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

5 Vì Sao Preprocessing Quan Trọng?

- ✓ Cải thiện độ chính xác và ổn định của model.
- ✓ Tăng tốc quá trình training và hội tụ.
- ✓ Giúp thuật toán học pattern có ý nghĩa.
- ✓ Ngăn overfitting do dữ liệu nhiễu.
- ✓ Bước thiết yếu để xây dựng model ML đáng tin cậy.

Lỗi Thường Gặp

- ✗ Preprocessing trước khi split dữ liệu.
- ✗ Không xử lý outliers và missing values.
- ✗ Dùng thông tin từ test data trong training.
- ✗ Encode sai categorical variables.
- ✗ Không scale feature cho model dựa trên khoảng cách.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Exploratory Data Analysis (EDA)

5

EDA là quá trình khám phá và tìm hiểu dữ liệu để phát hiện pattern, kiểm tra chất lượng, kiểm định giả thuyết và trích xuất insight hữu ích trước khi xây dựng model.

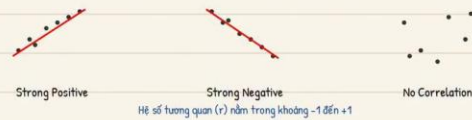
1 Các Đại Lượng Thống Kê Chính

Đại Lượng	Công Thức / Ý Nghĩa
Mean ($\bar{x}$)	Trung bình cộng tất cả giá trị.
Median	Giá trị ở giữa khi dữ liệu đã sắp xếp.
Mode	Giá trị xuất hiện nhiều nhất.
Variance (σ^2)	Bình phương độ lệch trung bình.
Std Deviation (σ)	Độ phân tán của dữ liệu.

2 Distributions (Hình Dạng Dữ Liệu)



3 Correlation (Mối Quan Hệ Giữa Feature)



4 Kỹ Thuật Visualization

- Histogram**
Thế hiện phân phối tần suất của continuous data.
- Box Plot**
Thế hiện median, quartile, độ phân tán và outlier.
- Scatter Plot**
Thế hiện mối quan hệ giữa hai numerical feature.
- Heatmap**
Trực quan hoá correlation giữa nhiều feature.
- Pie Chart**
Thế hiện tỉ lệ giữa các category.

5 Pattern & Anomaly

Pattern

- ✓ Trend (tăng/giảm theo thời gian)
- ✓ Seasonality (lặp lại theo chu kỳ)
- ✓ Clusters (nhóm tự nhiên)
- ✓ Relationships (tương tác giữa feature)

Anomaly / Outlier

- ✗ Giá trị nằm xa khoảng bình thường
- ✗ Có thể là lỗi hoặc sự kiện hiếm
- ✗ Phát hiện bằng IQR, Z-Score
- ✗ Xử lý bằng domain knowledge

Ghi Nhớ

Outlier không phải lúc nào cũng xấu. Chúng có thể là những insight giá trị!

6 Quy Trình EDA

- 1. Hiểu Dữ Liệu**
Load dataset, kiểm tra shape, kiểu dữ liệu.
- 2. Kiểm Tra Chất Lượng**
Missing values, duplicates, outliers.
- 3. Univariate**
Distribution, summary stats, visualization.
- 4. Bivariate**
Correlation, so sánh nhóm, quan hệ.
- 5. Insight & Hành Động**
Key findings, giả thuyết, bước tiếp theo.

✓ Best Practices

- Hiểu dữ liệu trước khi modeling.
- Dùng visualization cùng với thống kê.
- Xử lý missing values & outliers cẩn thận.
- Ghi chú lại insight và giả thuyết.

💡 Mẹo Phòng Vấn

- ☆ Luôn giải thích dữ liệu đang "nói" điều gì.
- ☆ Nhớ tên tool: Pandas, NumPy, Matplotlib, Seaborn.
- ☆ Nếu rõ EDA đã giúp modeling tốt hơn thế nào.

✗ Lỗi Thường Gặp

- ✗ Bỏ qua EDA và modeling luôn.
- ✗ Bỏ qua missing values & outliers.
- ✗ Chỉ dựa vào accuracy để đánh giá.

Kiến thức chất lượng, sự nghiệp tốt hơn!

Supervised Learning

6

Trong Supervised Learning, ta học một hàm ánh xạ từ input (feature) sang output (target) dựa trên labeled data.

1 Supervised Learning Là Gì?

- Sử dụng labeled data (feature + target).
- Học một hàm $f(x)$ ánh xạ input sang output đúng.
- Mục tiêu:** Tổng quát hoá từ các ví dụ đã biết sang dữ liệu chưa từng thấy.

Features (X)	Target (y)
...	10
...	25
...	15
...	30
...	...

2 Regression vs Classification

Regression (Output Liên Tục)



Dự đoán một giá trị số liên tục.

Ví dụ:

- Dự đoán giá nhà
- Dự báo nhiệt độ
- Dự đoán doanh số

Classification (Output Rời Rạc)



Dự đoán một class hoặc category rời rạc.

Ví dụ:

- Spam vs Not Spam
- Fraud vs Legit
- Cat vs Dog

Khác Biệt Chính

Regression → "Bao nhiêu?" (giá trị liên tục)

Classification → "Thuộc lớp nào?" (nhãn rời rạc)

Target Variable (y)

Regression → Numeric (vd: 3.14, 100, -5)

Classification → Categorical (vd: 0/1, Yes/No, A/B)

3 Xây Dựng Bài Toán

- Define Objective:** Ta đang muốn dự đoán điều gì?
- Collect Data:** Thu thập dữ liệu liên quan với label đúng.
- Define Features:** Chọn hoặc tạo feature có tính thông tin.
- Define Target:** Quyết định target variable (y).
- Split Data:** Train / Validation / Test.
- Choose Metric:** Chọn metric đánh giá.
- Select Algorithm:** Chọn thuật toán phù hợp.
- Train Model:** Học pattern từ training data.
- Evaluate Model:** Validate & test hiệu suất.
- Deploy Model:** Dùng model để dự đoán thực tế.

4 Quy Trình Thông Thường



Collect Data



Preprocess Data



70%/15%/15% Split



Train Model



Evaluate Model



Deploy Model

Tip

Luôn ngăn data leakage! Thông tin từ test data không bao giờ được dùng trong lúc training.

5 Thuật Toán Phổ Biến

Regression Algorithms

- ✓ Linear Regression
- ✓ Ridge Regression
- ✓ Lasso Regression
- ✓ Polynomial Regression
- ✓ Decision Tree Regressor
- ✓ Random Forest Regressor
- ✓ Gradient Boosting (XGBoost, LightGBM, CatBoost)

Classification Algorithms

- ✓ Logistic Regression
- ✓ K-Nearest Neighbors (KNN)
- ✓ Naive Bayes
- ✓ Decision Tree Classifier
- ✓ Random Forest Classifier
- ✓ Support Vector Machine (SVM)
- ✓ Gradient Boosting (XGBoost, LightGBM, CatBoost)

Ghi Nhớ

Dữ liệu tốt hơn + Feature tốt + Model đúng = Dự đoán chính xác!

Mẹo Phòng Vấn

Sẵn sàng giải thích khi nào dùng Regression vs Classification kèm ví dụ.

Lỗi Thường Gặp

- X Sai target variable
- X Data leakage
- X Dùng accuracy cho regression
- X Bỏ qua validation
- X Bỏ qua class imbalance

6 Ứng Dụng Thực Tế

Regression	→	Dự đoán giá nhà
Regression	→	Dự báo doanh số
Regression	→	Dự đoán thời tiết
Classification	→	Phát hiện spam
Classification	→	Phát hiện gian lận
Classification	→	Phân loại ảnh

Kiến thức chất lượng, sự nghiệp tốt hơn!



Các Thuật Toán Regression

7

Regression dự đoán một giá trị số liên tục. Ta mô hình hoá mối quan hệ giữa feature (X) và target (y).

1 Các Loại Thuật Toán Regression

Linear Regression  Mô hình quan hệ tuyến tính giữa X và y. $y = \beta_0 + \beta_1 x + \epsilon$ Use Case: Dự đoán giá, dự báo doanh số.	Multiple Regression  Dùng nhiều feature để dự đoán target. $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$ Use Case: Dự đoán giá nhà, lương.	Polynomial Regression  Mô hình quan hệ phi tuyến bằng polynomial feature. $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \epsilon \text{ (degree d)}$ Use Case: Dự báo nhu cầu, growth modeling.	Ridge Regression (L2)  Thêm L2 penalty để giảm overfitting bằng cách co hệ số lại. $J = \text{MSE} + \lambda \sum \beta_i^2$ Use Case: Khi có nhiều feature hoặc multicollinear.	Lasso Regression (L1)  Thêm L1 penalty, có thể đưa hệ số về đúng 0 (feature selection). $J = \text{MSE} + \lambda \sum \beta_i $ Use Case: Feature selection, high-dimensional data.
---	---	--	--	---

2 Giả Định Thường Gặp (Linear Regression)

- ✓ **Linearity:** Quan hệ giữa X và y là tuyến tính.
- ✓ **Independence:** Các quan sát độc lập nhau.
- ✓ **Homoscedasticity:** Phương sai của error không đổi.
- ✓ **Normality:** Error phân phối chuẩn.
- ✓ **No Multicollinearity:** Feature không tương quan mạnh với nhau.

3 Bảng So Sánh Tổng Quan

Thuật Toán	Loại Model	Phi Tuyến	Regularization	Feature Selection	Kiểm Soát Overfitting
Linear Regression	Linear	X	None	X	Thấp
Multiple Regression	Linear	X	None	X	Thấp
Polynomial Regression	Non-Linear	✓	None	X	Trung bình
Ridge Regression	Linear	X	L2	X	Cao
Lasso Regression	Linear	X	L1	✓	Cao

4 Ví Dụ Python (Linear Regression)

```

from sklearn.linear_model import
LinearRegression
import pandas as pd

data = pd.read_csv('house_prices.csv')
X = data[['area', 'bedrooms']] # features
y = data['price'] # target

model = LinearRegression()
model.fit(X, y)
pred = model.predict([[1200, 3]])
print(pred)

```

Note

Chọn đúng thuật toán dựa trên pattern dữ liệu và loại bài toán.

Mẹo Phòng Vấn

- X Linear regression đơn giản nhưng cần kiểm tra giả định.
- X Ridge giúp xử lý multicollinearity.
- X Lasso giúp feature selection.

5 Ứng Dụng Thực Tế

- 🏠 Dự đoán giá nhà
- 💰 Dự báo doanh số & doanh thu
- 📈 Dự báo nhu cầu
- 📦 Dự đoán lương
- 🌡️ Dự đoán nhiệt độ
- ❤️ Dự đoán điểm rủi ro sức khỏe

Lỗi Thường Gặp

- X Bỏ qua outlier và điểm ảnh hưởng lớn.
- X Dùng regression cho bài toán classification.
- X Không scale feature trước khi regularize.
- X Overfitting với polynomial bậc cao.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Các Thuật Toán Classification

8

Classification dự đoán một nhãn (class) rời rạc cho dữ liệu đầu vào. Thuật toán học một decision boundary để phân tách các class.

① Các Thuật Toán Classification Phổ Biến

① Logistic Regression


Môdel tuyến tính dùng cho binary classification. Output xác suất qua sigmoid function.

② K-Nearest Neighbors (KNN)


Thuật toán non-parametric. Class của một điểm quyết định bởi class đa số của K neighbor gần nhất.

③ Decision Tree


Môdel dạng cây, chia dữ liệu dựa trên điều kiện feature.

④ Random Forest


Ensemble của nhiều decision tree. Giảm overfitting và tăng độ chính xác.

⑤ Naive Bayes

$$P(A|B) = P(B|A)P(A)/P(B)$$

Thuật toán xác suất dựa trên Bayes' Theorem với giả định độc lập. Hiệu quả cho text, spam, NLP.

⑥ Support Vector Machine (SVM)


Tìm hyperplane tối ưu để maximize margin giữa các class. Hiệu quả với dữ liệu chiều cao.

② So Sánh Thuật Toán

Thuật Toán	Loại Model	Decision Boundary	Phù Hợp Khi	Ưu Điểm	Nhược Điểm
Logistic Regression	Linear	Linear	Quan hệ tuyến tính, dữ liệu nhỏ/vừa	✓ Đơn giản, nhanh, dễ diễn giải	✗ Underfit dữ liệu phức tạp
KNN	Non-Parametric	Non-Linear	Dữ liệu nhỏ, boundary phi tuyến	✓ Đơn giản, không cần training	✗ Predict chậm, nhạy với K
Decision Tree	Non-Linear	Non-Linear	Dữ liệu hỗn hợp, dễ diễn giải	✓ Dễ hiểu, xử lý phi tuyến	✗ Overfitting, phương sai cao
Random Forest	Ensemble	Non-Linear	Dataset lớn, phương sai cao	✓ Chính xác cao, ổn định	✗ Khó diễn giải hơn
Naive Bayes	Probabilistic	Linear/Non-Linear	Text data, chiều cao, thưa	✓ Nhanh, hiệu quả dữ liệu nhỏ	✗ Giả định độc lập
SVM	Linear/Non-Linear	Linear/Non-Linear	Chiều cao, margin rõ ràng	✓ Hiệu quả ở chiều cao	✗ Chậm với dữ liệu lớn

★ Điểm Chính

- ✓ Chọn thuật toán dựa trên dữ liệu, bài toán và yêu cầu business.
- ✓ Không có thuật toán nào là tốt nhất cho mọi bài toán.
- ✓ Thử nhiều model và so sánh hiệu suất.
- ✓ Validate bằng metric và test data phù hợp.

💡 Mẹo Phòng Vấn

- ☆ Hiểu rõ intuition, điểm mạnh, hạn chế, use case.
- ☆ Sẵn sàng giải thích decision boundary và kỹ thuật kiểm soát overfitting.
- ☆ Hiểu rõ bias-variance trade-off.

⚠️ Lỗi Thường Gặp

- ✗ Chỉ dùng accuracy để đánh giá.
- ✗ Bỏ qua feature scaling cho KNN & SVM.
- ✗ Không xử lý class imbalance.
- ✗ Bỏ qua cross-validation.

Ứng Dụng Thực Tế: ✉ Phát hiện Spam | 🛡 Phát hiện gian lận | 👤 Dự đoán churn khách hàng | ❤️ Chẩn đoán y khoa | 📷 Phân loại ảnh

Kiến thức chất lượng, sự nghiệp tốt hơn!

Unsupervised Learning

9

Unsupervised Learning tìm pattern hoặc cấu trúc ẩn trong dữ liệu mà không sử dụng bất kỳ nhãn (target) nào.

① Khái Niệm Clustering

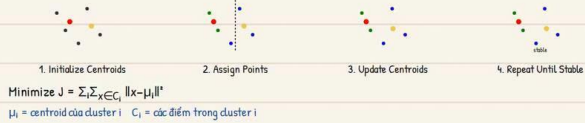
- Nhóm các data point tương tự nhau dựa trên feature.
- Các điểm trong cùng cluster tương tự nhau hơn.
- Không cần target variable.

💡 Use Cases

- ✓ Customer Segmentation
- ✓ Market Basket Analysis
- ✓ Document Grouping
- ✓ Image Segmentation

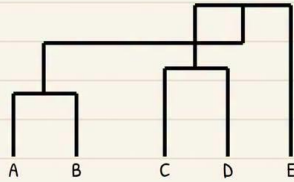
② K-Means Clustering

- Phân dữ liệu thành K cluster.
- Gán mỗi điểm cho centroid gần nhất.
- Cập nhật centroid = mean của các điểm.
- Lặp lại cho đến khi hội tụ.



③ Hierarchical Clustering

- Xây dựng hệ thống phân cấp cluster.
- Hai Loại: **Agglomerative** (Bottom-Up), **Divisive** (Top-Down)
- Trực quan hoá bằng Dendrogram.



④ DBSCAN

(Density-Based Spatial Clustering of Applications with Noise)

- Nhóm các điểm nằm gần nhau (mật độ cao).
- Nhận diện noise/outlier là các điểm ở vùng mật độ thấp.
- Tham số: **eps** (bán kính neighborhood), **min_samples** (số điểm tối thiểu)

⑤ So Sánh Thuật Toán

Thuật Toán	Cách Tiếp Cận	Hình Dạng Cluster	Cần K?	Xử Lý Noise	Khả Năng Mở Rộng	Phù Hợp Nhất
K-Means	Centroid Based	Spherical (Convex)	Có	Không	Cao	Dataset lớn, cluster tách biệt rõ
Hierarchical	Tree Based	Bất kỳ hình dạng	Không	Không	Thấp	Dataset nhỏ, phân tích khám phá
DBSCAN	Density Based	Hình dạng bất kỳ	Không	Có	Trung bình	Cluster có noise, phát hiện outlier.

⑥ Ví Dụ Customer Segmentation

Income	Spending	Age
40k	2k	25
60k	5k	40
30k	1k	22
80k	6k	50

K-Means Clustering (K=3) →

- High Value Customers
- Potential Customers
- Low Value Customers

⑦ Anomaly Detection (Unsupervised)

Tìm các pattern hiếm không tuân theo hành vi bình thường.

Phương pháp: Isolation Forest, One-Class SVM, Autoencoders, DBSCAN (noise).

Use case: Fraud Detection, Fault Detection, Network Intrusion, Medical Diagnosis.

🌟 Mẹo Phòng Vấn

- ✧ K-Means đơn giản nhưng giả định cluster hình cầu.
- ✧ DBSCAN tốt nhất cho noise và hình dạng bất kỳ.
- ✧ Hierarchical cho ra hierarchy (dendrogram).

🌟 Best Practices

- ✓ Scale feature trước khi clustering.
- ✓ Chọn đúng thuật toán theo dữ liệu.
- ✓ Dùng domain knowledge để diễn giải cluster.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Feature Engineering

10

Feature Engineering là nghệ thuật tạo ra feature tốt hơn từ dữ liệu thô để cải thiện hiệu suất model, giảm overfitting và nắm bắt pattern.

1 Vì Sao Feature Engineering Quan Trọng?

- ✓ Cải thiện độ chính xác của model.
- ✓ Giảm thời gian training.
- ✓ Giúp model tổng quát hoá tốt hơn.
- ✓ Biến domain knowledge thành tín hiệu hữu ích.
- ✓ Đôi khi quan trọng hơn cả thuật toán!

★ Điểm Chính

Feature tốt >> Model tốt

- ✓ Biểu diễn dữ liệu tốt hơn
- ✓ Pattern mạnh hơn
- ✓ Tổng quát hoá tốt hơn

2 Quy Trình Feature Engineering



3 Các Loại Feature Engineering

1. Feature Creation

Tạo feature mới hữu ích từ feature có sẵn.

Ví dụ:

Total = Price x Qty
Discount(%) = Discount / Price

2. Feature Selection

Chọn feature quan trọng nhất và loại bỏ nhiều.

Phương pháp:

Correlation, Variance Threshold, Chi-Square, Mutual Information, Model Based (RF, L1)

3. Feature Transformation

Biến đổi feature để phù hợp hơn với model.

Ví dụ:

Log Transformation, Power Transformation, Date/Time Features, Aggregations

4. Encoding Categorical

Chuyển categorical variable sang dạng số.

Ví dụ:

Label Encoding, One-Hot Encoding, Ordinal Encoding

5. Feature Scaling

Scale feature số về cùng khoảng.

Phương pháp:

Normalization (0-1), Standardization (Z-score), Robust Scaling

4 Binning (Discretization)

Chuyển dữ liệu liên tục thành các bin. Giúp xử lý pattern phi tuyến và outlier.

Age	Bin
0-18	Young
18-35	Adult
35-60	Middle Aged
60+	Senior

5 Interaction Features

Kết hợp 2 hoặc nhiều feature để nắm bắt pattern ẩn.

Income	Spending	Income x Spending
50000	20000	1,000,000
60000	25000	1,500,000

6 Dùng Domain Knowledge

Domain knowledge giúp:

- ✓ Xác định feature quan trọng
- ✓ Tạo feature có ý nghĩa
- ✓ Loại bỏ thông tin không liên quan

Ví dụ (E-commerce):

- Kết hợp category & giá sản phẩm.
- Ngày trong tuần, holiday flag, mùa.

7 Ví Dụ Python

```

import pandas as pd
df = pd.read_csv('sales.csv')

# Feature Creation
df['Total'] = df['Price'] * df['Quantity']

# Binning
df['Age_Group'] = pd.cut(df['Age'],
    bins=[0,18,35,60,100],
    labels=['Young', 'Adult', 'Middle Aged', 'Senior'])

# One-Hot Encoding
df = pd.get_dummies(df, columns=['City'], drop_first=True)
  
```

★ Mẹo Phòng Vấn

- Giải thích LÝ DO một feature hữu ích.
- Nói về tác động lên model.
- Luôn validate bằng thực nghiệm.

★ Best Practices

- Hiểu dữ liệu trước khi tạo feature.
- Tránh data leakage khi tạo feature.
- Kiểm tra multicollinearity.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Các Chỉ Số Đánh Giá Mô Hình

11

Các chỉ số đánh giá giúp ta đo lường model hoạt động tốt như thế nào trên dữ liệu chưa từng thấy và đưa ra quyết định đúng.

1 Confusion Matrix (Cho Classification)

	Positive (1)	Negative (0)
Positive (1)	TP (True Positive)	FP (False Positive)
Negative (0)	FN (False Negative)	TN (True Negative)

TP: Dự đoán positive, thực tế positive. FP: Dự đoán positive, thực tế negative.
FN: Dự đoán negative, thực tế positive. TN: Dự đoán negative, thực tế negative.

2 Classification Metrics

Accuracy = $(TP+TN)/(TP+TN+FP+FN)$ – Độ chính xác tổng thể.

Precision = $TP/(TP+FP)$ – Bao nhiêu dự đoán positive là đúng.

Recall (Sensitivity) = $TP/(TP+FN)$ – Bắt được bao nhiêu positive thực tế.

F1 Score = $2 \times (Precision \times Recall) / (Precision + Recall)$ – Cân bằng Precision & Recall.

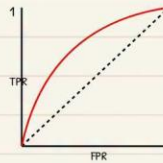
3 Ví Dụ Confusion Matrix

	Yes (1)	No (0)
Yes (1)	50	10
No (0)	5	35

TP=50, FP=10, FN=5, TN=35

Accuracy=85% | Precision=83.33% | Recall=90.91% | F1=86.96%

4 ROC Curve & ROC-AUC



- ROC curve thể hiện trade-off giữa TPR và FPR.
- AUC (Area Under Curve) đo khả năng phân tách.
- AUC = 1 → Model hoàn hảo.
- AUC = 0.5 → đoán ngẫu nhiên.
- AUC càng cao càng tốt.

5 Regression Metrics

MAE (Mean Absolute Error) – Sai số dự đoán trung bình tuyệt đối.

MSE (Mean Squared Error) – Phạt nặng hơn với sai số lớn.

RMSE (Root Mean Squared Error) – Cùng đơn vị với target, dễ diễn giải hơn.

R² (R-Squared) – Tỷ lệ phương sai được model giải thích.

6 So Sánh Metric

Metric	Dùng Tốt Cho	Đo Lường	Không	Tốt Hơn
Accuracy	Balanced Classes	Độ chính xác tổng thể	0-1	Cao hơn
Precision	High FP cost	Dự đoán positive đúng	0-1	Cao hơn
Recall	High FN cost	Bắt positive thực tế	0-1	Cao hơn
F1 Score	Imbalanced Classes	Cân bằng P & R	0-1	Cao hơn
ROC-AUC	Probabilistic Models	Khả năng phân tách	0-1	Cao hơn
MAE / MSE / RMSE	Regression	Sai số trung bình	0-∞	Thấp hơn
R ² Score	Regression	Phương sai giải thích	-∞ đến 1	Cao hơn

★ Chọn Đúng Metric

- ✓ Hiểu mục tiêu business và chi phí sai số.
- ✓ Kiểm tra phân phối class (balanced vs imbalanced).
- ✓ Dùng Precision/Recall/F1 cho dữ liệu mất cân bằng.
- ✓ Dùng ROC-AUC để so sánh model xác suất.
- ✓ Dùng MAE/RMSE cho bài toán regression.

🔥 Ứng Dụng Thực Tế

- 👉 Fraud Detection → Recall, ROC-AUC
- 👉 Disease Prediction → Recall, F1 Score
- 👉 Recommendation → Precision, AUC
- 👉 House Price Prediction → RMSE, MAE, R²

⚠️ Lỗi Thường Gặp

- ✗ Dùng Accuracy cho dữ liệu mất cân bằng.
- ✗ Bỏ qua chi phí business của sai số.
- ✗ Data leakage khi đánh giá.
- ✗ Không đặt baseline hợp lý.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Cross-Validation & Hyperparameter Tuning

12

Cross-Validation giúp đánh giá hiệu suất model một cách đáng tin cậy. Hyperparameter Tuning tìm cấu hình tốt nhất cho model.

1 Parameter vs Hyperparameter

Parameter	Hyperparameter
Giá trị được model học từ training data.	Giá trị người dùng thiết lập trước khi training.
Thay đổi trong quá trình training.	Điều khiển quá trình training.
Và weight (w), bias (b) trong Linear Regression.	Và learning_rate, n_estimators, max_depth, C, k, alpha.

2 K-Fold Cross-Validation

Chia dữ liệu thành K phần. Dùng K-1 phần để train, 1 phần để validate. Lặp lại K lần rồi lấy trung bình score.

Fold 1:	V	T	T	T	T
Fold 2:	T	V	T	T	T
Fold K:	T	T	T	T	V

V = Validation Set T = Training Set

Final Score = Trung bình của K validation score

3 Stratified K-Fold

Giữ nguyên tỉ lệ phân phối class trong mỗi fold. Quan trọng cho bài toán classification mất cân bằng.

Ví dụ (Imbalanced Data): Class 1: 90% - Class 0: 10%. Mỗi fold sẽ có tỉ lệ tương tự 90:10.

Khi Nào Dùng?

- ✓ Classification với class mất cân bằng.
- ✓ Khi phân phối target quan trọng.

4 Các Chiến Lược Cross-Validation

Chiến Lược	Dùng Cho	Ưu Điểm	Nhược Điểm
K-Fold CV	Bài toán tổng quát	Đơn giản, giảm phương sai	Có thể không giữ tỉ lệ class
Stratified K-Fold CV	Classification mất cân bằng	Giữ phân phối class	Phức tạp hơn một chút
Leave-One-Out (LOOCV)	Dataset nhỏ	Bias thấp	Phương sai cao, tốn tính toán
Time Series Split	Dữ liệu time series	Tôn trọng thứ tự thời gian	Ít dữ liệu training hơn

5 Phương Pháp Hyperparameter Tuning

Grid Search

Thử tất cả tổ hợp hyperparameter có thể. Tìm kiếm toàn diện (exhaustive).

```
params = {'n_estimators':  
[50, 100, 200], 'max_depth':  
[3, 5, 7]}  
Tổng model = 3 x 3 = 9
```

Random Search

Thử ngẫu nhiên các tổ hợp trong không gian tham số. Hiệu quả hơn và không gian lớn.

```
params = {'n_estimators':  
[50, 100, 200], 'max_depth':  
[3, 5, 7]}  
Tổng model = 5 (random  
picks)
```

6 Chiến Lược Validation Trong Phát Triển Model



7 Ví Dụ Python (GridSearchCV)

```
from sklearn.model_selection import GridSearchCV, KFold  
from sklearn.ensemble import RandomForestClassifier  
  
param_grid = {'n_estimators': [100, 200], 'max_depth': [None, 10, 20],  
              'min_samples_split': [2, 5]}  
cv = KFold(n_splits=5, shuffle=True, random_state=42)  
model = RandomForestClassifier(random_state=42)  
grid = GridSearchCV(model, param_grid, cv=cv, scoring='f1')  
grid.fit(X_train, y_train)  
print("Best Params:", grid.best_params_)  
print("Best CV Score:", grid.best_score_)
```

★ Interview Facts

- ✪ Cross-validation giảm phương sai, cho ước lượng ổn định.
- ✪ Stratified K-Fold bắt buộc cho classification mất cân bằng.
- ✪ Random Search nhanh hơn Grid Search với không gian lớn.

★ Best Practices

- ✓ Luôn dùng CV để đánh giá bên ngoài.
- ✓ Tránh data leakage khi CV.
- ✓ Chỉ đánh giá trên test set chưa từng thấy.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Overfitting, Underfitting & Bias-Variance

13

Một model tốt cân tổng quát hoá tốt trên dữ liệu chưa từng thấy. Nó cần có sự cân bằng đúng giữa Bias và Variance.

1. Khái Niệm Chính

- Bias:** Sai số do model đưa ra giả định quá đơn giản.
- Variance:** Sai số do model quá nhạy với biến động nhỏ của training data.
- Model Complexity điều khiển bias-variance trade-off.
- Mục tiêu:** Tối thiểu hoá tổng sai số để đạt tổng quát hoá tốt.

2. Underfitting (High Bias)



- Bias cao, variance thấp.
- Không học được từ dữ liệu.
- Training error cao & test error cao.
- Vd: model tuyến tính cho dữ liệu phi tuyến phức tạp.

3. Overfitting (High Variance)



- Bias thấp, variance cao.
- Học cả noise trong training data.
- Training error thấp & test error cao.
- Vd: polynomial bậc cao, cây quá sâu.

4. Good Fit (Balanced)



- Bias thấp, variance thấp (cân bằng).
- Nắm bắt pattern cốt lõi.
- Training & test error đều thấp.
- Tổng quát hoá tốt nhất trên dữ liệu mới.

5. Bias-Variance Trade-off



- Complexity thấp $\rightarrow$ Bias cao, Variance thấp.
- Complexity cao $\rightarrow$ Bias thấp, Variance cao.
- Chọn complexity tối ưu để tổng sai số nhỏ nhất.

6. Learning Curves



7. Nguyên Nhân & Giải Pháp

Vấn Đề	Nguyên Nhân	Triệu Chứng	Giải Pháp
Overfitting (High Variance)	Model quá phức tạp, quá nhiều feature, dataset nhỏ/nhiều	Training error thấp, validation error cao	Regularization (L1/L2), giảm feature, thêm dữ liệu, CV, early stopping
Underfitting (High Bias)	Model quá đơn giản, thiếu feature quan trọng	Training error cao, validation error cao	Tăng độ phức tạp, thêm feature, train lâu hơn
Good Fit (Balanced)	Dữ liệu tập đúng, đủ dữ liệu	Training & validation error thấp	Giữ nguyên cách tiếp cận, theo dõi validation

8. Regularization

Thêm penalty để giảm độ phức tạp model.

L2 (Ridge): $J = \text{MSE} + \lambda \sum w_i^2$

L1 (Lasso): $J = \text{MSE} + \lambda \sum |w_i|$

$\lambda \rightarrow$ độ mạnh regularization (λ cao hơn = penalty mạnh hơn)

Mẹo Thực Tế

- ✓ Luôn vẽ learning curve.
- ✓ Dùng cross-validation.
- ✓ Thử model đơn giản trước, tăng dần độ phức tạp.

Interview Facts

- ☹ No free lunch theorem $\rightarrow$ không model nào tốt nhất cho mọi bài toán.
- ☹ Hướng tới bias-variance cân bằng.

Lỗi Thường Gặp

- ✗ Dùng model quá phức tạp cho dữ liệu nhỏ.
- ✗ Bỏ qua validation set.
- ✗ Over-regularizing (λ quá cao).

Kiến thức chất lượng, sự nghiệp tốt hơn!



ML Nâng Cao & Ensemble Learning

14

Ensemble Learning kết hợp nhiều model để đạt độ chính xác, ổn định và khả năng tổng quát hoá tốt hơn một model đơn lẻ.

1 Các Paradigm Ensemble Learning

A. Bagging
Giảm variance bằng cách train các model song song trên các mẫu khác nhau.
Ví dụ: Random Forest

B. Boosting
Giảm bias bằng cách train các model tuần tự, tập trung vào lỗi trước đó.
Ví dụ: AdaBoost, GBM, XGBoost

C. Stacking
Kết hợp các model khác nhau bằng một model khác (meta-learner).
Ví dụ: Stacking Classifier

D. Voting
Kết hợp dự đoán bằng majority hoặc averaging đơn giản.
Ví dụ: Hard & Soft Voting

2 Thuật Toán Ensemble Phổ Biến

Thuật Toán	Loại	Ý Tưởng Chính	Điểm Mạnh
Random Forest	Bagging	Nhiều Decision Tree trên bootstrap sample + random feature	Giảm overfitting, tốt với tabular data
Gradient Boosting	Boosting	Cây huấn luyện mới xây dựa vào lỗi của cây trước đó	Độ chính xác cao, xử lý pattern phức tạp
XGBoost	Boosting	GBM kết hợp với regularization, xử lý missing values	Nhanh, scalable, xử lý missing values
LightGBM	Boosting	Leaf-wise tree growth, dựa trên histogram	Rất nhanh & tiết kiệm bộ nhớ
CatBoost	Boosting	Xử lý categorical feature bằng ordered boosting	Xuất sắc với dữ liệu categorical

3 Khi Nào Ensemble Vượt Trội?

- ✓ Quan hệ phi tuyến, phức tạp
- ✓ Dữ liệu chiều cao
- ✓ Dataset nhiều hoặc mất cân bằng
- ✓ Cần độ chính xác dự đoán cao
- ✓ Model đơn lẻ có variance cao (như Decision Tree)

★ Điểm Chính

Ensemble kết hợp "trí tuệ" của nhiều model yếu/mạnh để tạo ra một model mạnh, ổn định và chính xác hơn.

4 Bỏ Trọ Bias-Variance Trade-off

Bagging ↓ Variance | Boosting ↓ Bias | Stacking → Best of both worlds

5 Ví Dụ Python (Random Forest)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
model = RandomForestClassifier(n_estimators=100, max_depth=None, random_state=42)
model.fit(X, y)
print("Accuracy:", model.score(X, y))
```

6 So Sánh Nhanh

Khía Cạnh	Bagging (RF)	Boosting (GBM/XGB/LGBM/CatBoost)	Stacking	Voting
Training	Song song	Tuần tự	Hai tầng	Song song
Giảm	Variance	Bias (và Variance)	Cả hai	Variance
Độ Phức Tạp	Trung bình	Cao	Cao	Thấp
Tốc Độ	Nhanh	Chậm hơn	Chậm hơn	Nhanh
Dùng Khi	Overfitting	Underfitting	Cần tổng quát hoá tốt nhất	Đơn giản & hiệu quả

💡 Mẹo Phòng Vấn

- Bagging = Song song → Giảm Variance
- Boosting = Tuần tự → Giảm Bias
- Stacking = Kết hợp nhiều model khác loại
- Ensemble hầu như luôn vượt trội model đơn lẻ trên dữ liệu thực tế

★ Best Practices

- ✓ Tune hyperparameter cẩn thận.
- ✓ Dùng cross-validation.
- ✓ Tránh data leakage.
- ✓ Phân tích feature importance.
- ✓ Dùng early stopping khi boosting.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Giảm Chiều Dữ Liệu & Model Interpretability

15

Giảm chiều dữ liệu (Dimensionality Reduction) đơn giản hoá dữ liệu nhưng vẫn giữ thông tin quan trọng. Model Interpretability giúp ta hiểu vì sao model đưa ra một dự đoán.

① Vì Sao Cần Giảm Chiều?

- ✓ Dữ liệu chiều cao làm tăng độ phức tạp và thời gian training.
- ✓ Nhiều feature có thể dư thừa hoặc nhiễu.
- ✓ Giúp trực quan hoá (2D/3D).
- ✓ Cải thiện hiệu suất và khả năng tổng quát hoá.

② PCA (Principal Component Analysis)

- Biến đổi feature tương quan gốc thành component không tương quan (PC).
- PC1 nắm giữ variance lớn nhất, PC2 tiếp theo, ...
- Giúp giảm chiều mà vẫn giữ thông tin.

Cách PCA Hoạt Động?

1. Standardize dữ liệu
2. Tính covariance matrix
3. Tính eigenvalue & eigenvector
4. Chọn top % PC
5. Transform dữ liệu

③ Feature Selection vs Dimensionality Reduction

Khía Cạnh	Feature Selection	Dimensionality Reduction
Mục Tiêu	Chọn tập con feature gốc	Tạo feature mới (tổ hợp)
Khả Năng Diễn Giải	Cao	Thấp hơn (PC là tổ hợp)
Mất Thông Tin	Ít nhất	Có thể mất một phần
Ví Dụ	Filter (Corr, Chi ²), Wrapper (RFE), Embedded (Lasso)	PCA, t-SNE, UMAP, Autoencoders
Phù Hợp Nhất	Khi cần model đơn giản, dễ diễn giải	Khi feature tương quan cao hoặc chiều rất cao

④ Kỹ Thuật Giảm Chiều Phổ Biến

Kỹ Thuật	Loại	Use Case
PCA	Linear	Tổng quát, giảm feature tương quan
t-SNE	Non-linear	Trực quan hoá dữ liệu chiều cao
UMAP	Non-linear	Trực quan hoá + nhanh hơn t-SNE
Autoencoders	Non-linear	Học biểu diễn nén (Deep Learning)

PCA Giải Thích

PCA tìm trục mới (PC) nắm giữ variance lớn nhất. Ta giữ lại top % PC giải thích phần lớn variance.
Explained Variance Ratio = Variance của PC được chọn / Tổng Variance

⑤ Feature Importance

- ✓ Cho biết mỗi feature đóng góp bao nhiêu vào dự đoán.
- ✓ Giúp hiểu model, debug, và feature selection.
- ✓ Tree-based model có sẵn importance score.

⑥ SHAP (SHapley Additive exPlanations)

- ✓ Dựa trên game theory (Shapley values).
- ✓ Giải thích từng dự đoán riêng lẻ.
- ✓ Thể hiện mỗi feature đóng góp dương (+) hay âm (-).
- ✓ Model-agnostic (dùng được với mọi model).
- ✓ Giải thích ở mức Global (tổng quan) & Local (từng dự đoán).

⑦ Ví Dụ: SHAP Summary Plot

- Hô điểm = một sample. - Màu thể hiện giá trị feature (Cao → Đỏ, Thấp → Xanh). - Bên phải → tăng dự đoán. - Bên trái → giảm dự đoán.

⑧ Khi Nào Dùng Gì?

Tình Huống	Cách Tiếp Cận Đề Xuất
Nhiều feature tương quan	PCA
Cần feature dễ diễn giải	Feature Selection
Dữ liệu text/ảnh chiều cao	PCA / Autoencoders
Cần trực quan hoá (2D/3D)	t-SNE / UMAP
Giải thích một dự đoán	SHAP

★ Interview Facts

- ✖ PCA là kỹ thuật unsupervised.
- ✖ Feature Selection giữ nguyên feature gốc.
- ✖ Tổng SHAP value = (dự đoán - base value).

💡 Best Practices

- ✖ Luôn scale dữ liệu trước PCA.
- ✖ Tránh dùng quá nhiều feature → rủi ro overfitting.
- ✖ Dùng interpretability để tăng độ tin cậy của model.

⚠ Lỗi Thường Gặp

- ✖ Áp dụng PCA mà không scale.
- ✖ Dùng quá nhiều PC → giữ lại noise.
- ✖ Bỏ qua feature importance ở model phức tạp.

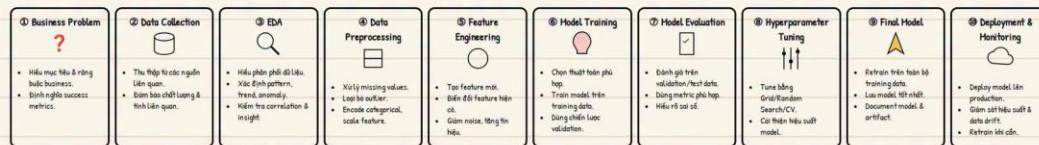
Kiến thức chất lượng, sự nghiệp tốt hơn!



Dự Án ML Hoàn Chỉnh Từ Đầu Đến Cuối

16

Một dự án ML thành công tuân theo quy trình có cấu trúc từ việc hiểu bài toán business đến việc triển khai một model sẵn sàng cho production và tạo ra giá trị.



↳ Lặp Lại & Cải Thiện Liên Tục

Hoạt Động Chi Tiết

- 1. **Business Problem:** Định nghĩa mục tiêu, target metric, phạm vi, ràng buộc.
- 2. **Data Collection:** Thu thập dữ liệu từ DB, API, file, log, khảo sát...
- 3. **EDA:** Trục quan hoá dữ liệu, phân tích phân phối, phát hiện missing values, correlation.
- 4. **Preprocessing:** Làm sạch dữ liệu, encode, scale, xử lý imbalance nếu có.
- 5. **Feature Engineering:** Thêm interaction, polynomial, feature theo domain.
- 6. **Model Training:** Thử nhiều thuật toán, xây baseline model.
- 7. **Evaluation:** So sánh hiệu suất bằng metric & CV phù hợp.
- 8. **Tuning:** Tối ưu hyperparameter để tổng quát hoá tốt nhất.
- 9. **Final Model:** Train model cuối, lưu pipeline & artifact.
- 10. **Deployment & Monitoring:** Serve model, giám sát metric & retrain.

Tỉ Lệ Split Dữ Liệu Thông Thường

Tập	Mục Đích	Tỉ Lệ Thường Dùng
Training Set	Học pattern từ dữ liệu	60% - 80%
Validation Set	Tune hyperparameter & chọn model	10% - 20%
Test Set	Đánh giá cuối, không thiên vị	10% - 20%

Metric Đánh Giá Thông Dụng

Classification	Regression
Accuracy, Precision, Recall, F1 Score, ROC-AUC, Log Loss	MAE, MSE, RMSE, R ² Score, MAPE

Ví Dụ Python Sklearn Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=200, random_state=42))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print('Test Accuracy :', pipeline.score(X_test, y_test))
```

Vì Sao Tuân Theo Quy Trình Này?

- ✓ Đảm bảo model chất lượng cao & đáng tin cậy.
- ✓ Cải thiện độ chính xác & khả năng tổng quát hoá.
- ✓ Dễ tái tạo và bảo trì.
- ✓ Giúp scale lên production.

Điểm Chính

Một pipeline ML được xây dựng tốt + dữ liệu tốt + đánh giá đúng = Model mạnh mẽ, đáng tin cậy, mang lại giá trị business thực sự.

Best Practices

- Luôn bắt đầu với mục tiêu business rõ ràng.
- Đánh nhiều thời gian hiểu dữ liệu.
- Ngăn data leakage mọi lúc.
- Theo dõi experiment & kết quả.
- Document mọi thứ để tái tạo.

Interview Facts

- ⚡ 80% thời gian ML dành cho chuẩn bị dữ liệu.
- Không có model hoàn hảo — chỉ có model đúng cho bài toán.
- ⚡ Chiến lược đánh giá tốt > model phức tạp.
- Luôn validate trên test data chưa từng thấy.

Lỗi Thường Gặp

- ✗ Không hiểu rõ bài toán business.
- ✗ Train và test trên cùng dữ liệu.
- ✗ Bỏ qua chất lượng & missing values.
- ✗ Overfitting do không dùng validation.
- ✗ Không giám sát model sau khi deploy.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Triển Khai ML & MLOps

17

MLOps là thực hành triển khai, giám sát và duy trì model ML trong production một cách đáng tin cậy và ở quy mô lớn.

① Model Serialization

Serialize (lưu) model đã train và các object liên quan để dùng sau này.

Dịnh dạng phổ biến:

joblib / pickle → Python objects
ONNX → Cross platform
SavedModel → TensorFlow
TorchScript → PyTorch

```
# Using joblib
import joblib
joblib.dump(model, 'model.pkl')
loaded_model = joblib.load('model.pkl')
```

② API Deployment

Expose model dưới dạng API để ứng dụng có thể gọi dữ liệu và nhận dự đoán.

FastAPI (Hiện đại & Nhanh)

```
from fastapi import FastAPI
import joblib, numpy as np
app = FastAPI()
model = joblib.load('model.pkl')
@app.post('/predict')
def predict(data: dict):
    x = np.array([list(data.values())])
    pred = model.predict(x)[0]
    return {'prediction': float(pred)}
```

Flask (Đơn giản & Nhẹ)

```
from flask import Flask, request, jsonify
import joblib, numpy as np
app = Flask(__name__)
model = joblib.load('model.pkl')
@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    x = np.array([list(data.values())])
    pred = model.predict(x)[0]
    return jsonify({'prediction': float(pred)})
```

③ Dockerizing Model

Đóng gói model + code + dependencies vào docker image để triển khai đồng nhất.

```
# Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Chạy Container
docker build -f ml-api
docker run -p 8080:8080 ml-api

④ CI/CD Pipeline Cho ML

Code Commit → Automated Build → Model Training → Model Validation → Build Docker Image → Deploy → Monitor
Push code lên Git repo → Cài deps, chạy test → Train model (optional) → Đánh giá metric, threshold → Tạo Docker image → Deploy staging/production → Log metric, drift

⑤ Model Monitoring

Giám sát gì?

- ✓ Prediction accuracy / metric
- ✓ Data drift
- ✓ Model drift
- ✓ Latency & throughput
- ✓ Error rate
- ✓ Tác động lên Business KPI

Công cụ giám sát: Prometheus + Grafana, Evidently AI, WhyLabs, MLflow, Amazon SageMaker Model Monitor

⑥ Data Drift vs Model Drift

Data Drift: Phân phối dữ liệu input thay đổi theo thời gian.
Model Drift: Hiệu suất model giảm dần theo thời gian.

⑦ Chiến Lược Retraining

- ✓ Phát hiện drift hoặc suy giảm hiệu suất
- ✓ Thu thập dữ liệu mới
- ✓ Label (nếu cần)
- ✓ Retrain model
- ✓ Validate model mới
- ✓ Deploy version mới
- ✓ Giám sát liên tục

⑧ Kiến Trúc MLOps (Tổng Quan)

Data Sources
DB, API, Logs

Data Pipeline
Ingest, Clean

Feature Store
Store & Serve

Training
Train & Validate

Model Registry
Version & Store

Serving (API)
Expose Model

Monitoring
Alert, Retrain

★ Điểm Chính

MLOps đảm bảo model ML đáng tin cậy, scalable và liên tục được cải thiện trong môi trường thực tế.

★ Best Practices

- ✓ Tự động hoá mọi thứ
- ✓ Version hoá dữ liệu & model
- ✓ Giám sát liên tục
- ✓ Bắt đầu đơn giản, lặp lại & cải thiện

💡 Interview Facts

Serialization → joblib, pickle, ONNX
API → FastAPI (async, nhanh), Flask (đơn giản)
Docker → Môi trường đồng nhất
CI/CD → Tự động build, test, deploy
Drift → Thay đổi phân phối dữ liệu/hiệu suất

⚠ Lỗi Thường Gặp

- ✗ Không giám sát sau khi deploy.
- ✗ Training-serving skew.
- ✗ Bỏ qua chất lượng dữ liệu.
- ✗ Không có chiến lược version/rollback.

Kiến thức chất lượng, sự nghiệp tốt hơn!



Ôn Tập Nhanh & Cẩm Nang Phòng Vấn ML

18

Một trang để ôn lại toàn bộ hành trình Machine Learning!

1 Quy Trình ML Hoàn Chỉnh



2 Cheat Sheet Chọn Thuật Toán

Loại Bài Toán	Thuật Toán	Phù Hợp Nhất Cho	Ghi Chú
Classification	Logistic Regression, DT, Random Forest, SVM, XGBoost, LightGBM	Spam detection, Churn prediction, Fraud detection	Dùng ROC-AUC, F1 cho dữ liệu mất cân bằng
Regression	Linear Regression, Ridge, Lasso, SVR, XGBoost, LightGBM	Price prediction, Demand forecasting	Dùng RMSE, MAE, R ²
Clustering	K-Means, Hierarchical, DBSCAN	Customer segmentation, Grouping	Không cần nhãn, dùng silhouette score
Dimensionality Reduction	PCA, t-SNE, UMAP, Autoencoders	Visualization, Giảm dư thừa	PCA cho linear, t-SNE cho non-linear
Anomaly Detection	Isolation Forest, One-Class SVM, Local Outlier Factor	Fraud, Network intrusion, Defect detection	Hoạt động trên pattern dữ liệu bình thường
Recommendation	Collaborative Filtering, Content Based, Matrix Factorization	Product recommendation, Movie/Item suggestions	Dùng implicit/explicit feedback

3 Metric & Công Thức Chính

A. Classification Metrics
Accuracy = $(TP+TN)/(TP+TN+FP+FN)$
Precision = $TP/(TP+FP)$
Recall (Sensitivity) = $TP/(TP+FN)$
F1 Score = $2 \times (Precision \times Recall) / (Precision + Recall)$
ROC-AUC = Diện tích dưới ROC Curve

B. Regression Metrics

MSE = $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$
RMSE = $\sqrt{MSE}$
R² = $1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

Ghi Nhớ

- ✓ Chọn metric theo mục tiêu business.
- ✓ Accuracy có thể gây hiểu nhầm và độ lệch mất cân bằng.
- ✓ RMSE/RMSE thấp hơn là tốt hơn.
- ✓ R² & ROC-AUC cao hơn là tốt hơn.

4 Câu Hỏi Phòng Vấn Thường Gặp

- Machine Learning là gì? Có mấy loại ML?
- Supervised vs Unsupervised vs Reinforcement Learning?
- Khác biệt giữa Parameter và Hyperparameter?
- Overfitting là gì và cách xử lý?
- Bias vs Variance trade-off?
- Gradient Descent hoạt động như thế nào?
- Cross-Validation là gì và vì sao cần dùng?
- Khác biệt giữa Normalization & Standardization?
- Giải thích Confusion Matrix và các metric.
- Khi nào dùng K-Means vs DBSCAN?
- Regularization là gì? (L1 vs L2)
- Ensemble Learning là gì? Có mấy loại?
- PCA là gì và hoạt động ra sao?
- Bạn sẽ deploy một ML model như thế nào?
- Cách xử lý Data Drift & Model Drift?
- Trình bày một dự án ML bạn đã làm.

5 Khung Trình Bày Dự Án

- 1 Problem Statement → Bài toán & mục tiêu business là gì?
- 2 Data Understanding → Nguồn, kích thước, feature, chất lượng.
- 3 EDA & Insights → Pattern chính, correlation, phát hiện trục trặc.
- 4 Preprocessing → Missing values, encoding, scaling, outlier.
- 5 Feature Engineering → Feature mới, selection, biến đổi.
- 6 Modeling → Thuật toán đã dùng và vì sao.
- 7 Evaluation → Metric, kết quả, so sánh.
- 8 Tuning → Chiến lược tune hyperparameter.
- 9 Final Model → Model tốt nhất, tóm tắt hiệu suất.
- 10 Deployment Impact → Giúp ích gì cho business, bước tiếp theo.

6 Checklist Khái Niệm Quan Trọng

- ✓ Data → Information → Knowledge
- ✓ Train / Validation / Test Split
- ✓ Feature Engineering là chìa khóa
- ✓ Ngăn Data Leakage
- ✓ Chọn Metric đúng cho bài toán
- ✓ Baseline Model trước
- ✓ Lập lại & Cải thiện
- ✓ Giám sát trong Production
- ✓ Retrain khi hiệu suất giảm
- ✓ Business Impact là mục tiêu cuối cùng

Học Phông Vốn

Luôn suy nghĩ thành tiếng, giữ thích giả định, và biến mình lười chọn bằng cả lý do kỹ thuật lẫn business.

7 Tóm Tắt Công Thức / Khái Niệm Nhanh

Gradient Descent Update:
 $\theta_{new} = \theta_{old} - \alpha \nabla J(\theta)$
(α = learning rate)

Sigmoid / Logistic Function:
 $\sigma(z) = \frac{1}{1+e^{-z}}$

Regularization (Cost Function):
 $J = MSE + \lambda \sum w_i^2$ (L2)
 $J = MSE + \lambda \sum |w_i|$ (L1)

Overfitting vs Underfitting	
Overfitting Bias thấp, Variance cao Train score cao Test score thấp Model quá phức tạp	Underfitting Bias cao, Variance thấp Train score thấp Test score thấp Model quá đơn giản
Balanced Cân bằng bias & variance Hiệu suất tốt trên train & test	

Điểm Chính

- Hiểu rõ bài toán một cách sâu sắc.
- Dữ liệu tốt + Feature tốt = Model mạnh.
- Đánh giá đúng cách và tune thông minh.
- Deploy có trách nhiệm và giám sát liên tục.
- ML là một quá trình lặp lại → Học, Cải thiện, Lặp lại!

Ghi Nhớ

Model chỉ là công cụ.
Giá trị đến từ việc giải quyết bài toán business thực sự!

Kiến thức chất lượng, sự nghiệp tốt hơn!